

Building the cross-border commerce & logistics rail for African botanicals

A complete explainer: what we built, what it does, how it works under the hood, who it serves, and the freight network that makes it defensible — plus a full account of the security hardening and design rebuild just completed.

46

BOTANICALS
CATALOGUED

74

COMPLIANCE TESTS
PASSING

4

SIDES OF THE
MARKETPLACE

0

KNOWN VULNERABILITIES
(BACKEND)

CONTENTS

Explainer

- 01 Executive summary
- 02 What we have created — the product
- 03 What we have done — the security & design programme
- 04 How it works — technical architecture
- 05 How it works — the compliance engine
- 06 How it works — payments
- 07 Customers — who buys
- 08 Clients — sellers, cargo agents & admin
- 09 Logistics — the moat
- 10 Business model & economics
- 11 Verification & evidence
- 12 What's next / open decisions
- 13 Appendix — full fix log

How to read this document. Sections 2–3 describe the business and the work completed. Sections 4–6 go under the hood for anyone technical. Sections 7–9 describe the people the platform serves. Section 13 is the detailed engineering record for anyone who wants line-by-line evidence.

What Ewe ati Egbo is, in one page

Ewe ati Egbo ("leaves and roots" in Yoruba) is a two-sided-becomes-four-sided platform that lets African producers and UK diaspora sellers list traditional herbal, botanical and natural-wellness products, and lets UK-based customers discover and buy them in their own vernacular — while a dedicated logistics layer, run by our own cargo arm and a network of onboarded third-party cargo agents, physically moves the goods from Africa to the customer's door.

It is deliberately **not** "an online shop for herbs." Two things make it a real business rather than a storefront:

1. **The freight network.** Anyone can clone a shopping cart. Almost nobody can cheaply clone a working, compliant, cost-effective Africa → UK freight corridor for a fragmented, low-trust supply chain of thousands of small sellers. That network is the asset a strategic acquirer would actually be buying.
2. **The compliance rail.** UK law treats a jar of dried leaves as food, supplement, cosmetic or medicine depending on what it's claimed to do — and getting that wrong is a regulatory and reputational risk. The platform enforces this automatically, at the point of listing, so individual sellers never have to understand MHRA/FSA/OPSS boundaries themselves.

Over this engineering cycle, the codebase went through two connected pieces of work:

Workstream	Summary
Security & correctness audit	Ten specialised review passes across the backend, database, compliance engine and mobile app surfaced 12 critical/high findings, a dozen medium-severity gaps, and 12 functional bugs in the mobile app. Nearly all of them have since been fixed and verified against a live database and a live server — not just reviewed on paper.
Full design rebuild	The mobile app's visual language, motion system and responsive behaviour were rebuilt from the ground up: a shared design-token system, spring-based animation throughout, and a layout engine that adapts the same screens across phone, tablet and desktop web — so the product now looks and feels like a funded consumer app rather than a prototype.

Where things stand today. The hardened, redesigned app lives on branch `fix/security-and-design`, two commits ahead of `main`, verified with 74 automated tests, 16 simulated database attacks (all blocked), and browser testing at four screen widths. Three decisions remain outside the scope of code — the payment settlement model, final regulatory sign-off on three herb safety flags, and UK GDPR paperwork — and are called out explicitly in Section 12 rather than silently assumed.

What we have created

The problem it solves

More than 2.5 million people of African and Caribbean heritage live in the UK, and they buy traditional herbs, botanicals and natural products regularly — for cooking, wellbeing and culture. Today that trade runs through an informal, low-trust patchwork: WhatsApp sellers, Instagram DMs, Etsy listings under the wrong name, suitcase importers, and "a cousin who knows someone." That patchwork is untrusted (no verification, inconsistent quality), unstructured (wrong or missing names, no ingredient or origin information, no compliance checking), and — most importantly — logistically painful, because the genuinely hard part is getting a product from a farm or market in Lagos, Accra or Nairobi to a customer's door in Leeds, reliably, legally and affordably.

The four sides of the platform

Actor	What they get
Customer (UK)	Vernacular search ("ewuro" resolves to bitter leaf; "onugbu" — the Igbo name for the same plant — resolves to the same product), trusted verified-seller listings, a multi-seller basket, an upfront cross-border delivery quote with carrier and ETA, order tracking leg-by-leg, and reviews.
Seller (Africa or UK-based diaspora)	Onboarding and verification, a storefront, a structured product-listing wizard with automatic compliance checking, order management, payouts, and a choice of fulfilment routes (our cargo network or their own).
Cargo agent	Onboarding, the ability to publish routes and rates on the corridors they serve, a consignment queue, shipment tracking updates, and payouts for the legs they carry.
Admin (platform operator)	Seller and product verification, claims/compliance moderation, import-rule configuration, routing oversight, order and dispute handling, and reporting.

The botanical catalogue

At the heart of the product is a proprietary dataset: **46 botanicals across 8 categories**, each entry carrying every vernacular name it is known by (Yoruba, Igbo, Hausa, English and botanical/Latin names), a UK regulatory classification, and a traffic-light compliance flag:

- **Green** — food/cosmetic use, no special restriction
- **Amber** — restricted or requires careful labelling (e.g. known active compounds, allergen or dosage concerns)

- **Red** — cannot be sold or marketed for the flagged use on the platform

This mapping — vernacular name to botanical identity to UK regulatory class — does not exist anywhere else in a structured, searchable form. It is as much a data moat as the freight network is a logistics moat.

The mobile app

Built on Expo (React Native) so the same codebase ships to iOS, Android and the web. Customers browse a home feed with featured products and tradition/category shortcuts, search and filter in Explore, view rich product detail pages with trust signals and a live delivery quote, manage a basket grouped by seller, place orders, and track them on an animated timeline from origin hub to their door.

What we have done

3.1 — The ten-agent audit

Ten specialised review passes were run in parallel across the entire codebase: backend API surface, the Postgres/Supabase schema and its row-level security policies, the compliance/claims engine, the shared search logic, the mobile app's security posture, its functional correctness, its accessibility, and its dependency health. The audit report (docs/security-audit-2026-09-23.md) recorded 12 critical/high findings, roughly a dozen medium-severity issues, 12 mobile functional bugs, and a short list of things that were already clean (no secrets in git history, no ReDoS, no prototype pollution, Helmet active, marketing copy already compliant).

The critical findings — and their fixes

#	Finding	Fix	Status
1	Paystack webhooks could be replayed to trigger duplicate payouts	Unique (provider, provider_ref) constraint, a dedicated webhook_events idempotency table, payout legs tied to a specific payment	Fixed
2	Order prices were trusted from the client, not priced server-side	Database now guards price columns from client edits; cart is labelled as an estimate	Partial
3	Split payments could pay sellers and cargo agents before delivery was confirmed	Schema now supports seller/agent/platform payout legs that cannot exceed the payment total; the settlement policy itself needs a business decision	Decision needed
4	Deleting an order cascaded and silently deleted its payment records	Changed to ON DELETE RESTRICT on every money/dispute table — financial history can no longer be wiped	Fixed
5	A user could edit their own role, verification status or payout account directly	Column-level database triggers block self-edits to privileged fields unless the write comes from a trusted server-side path	Fixed
6	users table wasn't linked to Supabase's own auth identity	Foreign key to auth.users plus a signup trigger that can never copy a "role" from user-supplied metadata (which would have let anyone self-promote to admin)	Fixed
7	A red-flagged (restricted) herb named only in free text — not the structured field — published anyway; an unrecognised herb silently defaulted to "safe"	The server now scans listing text itself for every herb in the dictionary and takes the worst flag found; an herb it cannot identify at all is sent to human review instead of defaulting to green	Fixed

8	The medical-claim scanner could be defeated by roughly 35 different bypass techniques — misspellings, vernacular phrasing, Cyrillic look-alike letters, zero-width characters, leetspeak, emoji	Full rewrite: Unicode normalisation, a homoglyph map, leetspeak detection, and word lists spanning English, Yoruba, Pidgin and Igbo phrasing for medical claims — covered by 74 automated regression tests	Fixed
9	Three herb safety flags looked too permissive for their known chemistry (pyrrolizidine alkaloids, toxic seed matter, caffeine content)	Re-flagged (see Section 5); each carries an explicit note that it is pending sign-off from a UK regulatory adviser	Pending adviser
10	A critical-severity vulnerability in an unused authentication dependency, plus high-severity issues in the web framework itself	Removed the unused dependency entirely and upgraded the framework across a major version; <code>npm audit</code> on the backend now reports zero vulnerabilities	Fixed
11	The signed-in session token was stored in plain text, readable by any script running on the same web origin, and included in Android's automatic app backups	Encrypted device keychain/keystore storage on phones, browser session storage (not persistent local storage) on web, backups disabled for the Android build	Fixed
12	A production build with missing configuration silently fell back to a fake "demo" mode — showing invented sample orders and a checkout button that looked real but did nothing	A production build now shows an explicit maintenance screen unless demo mode is deliberately switched on	Fixed

Everything else that was fixed

Beyond the 12 headline items: the backend no longer leaks stack traces on invalid input (it now returns a clean, structured 400 response); CORS is restricted to an allowlist instead of reflecting any origin; rate limiting is keyed correctly behind a proxy; the server refuses to start in production with a missing or placeholder secret; every status/role/class column in the database now has an enforced set of valid values; monetary columns are non-negative and currency-aware; foreign keys are indexed; a proof-of-delivery table and a tamper-evident, hash-chained audit log were added; the vernacular search engine's ambiguous-match guessing was replaced with an "ask, don't guess" rule; and all 12 of the mobile app's functional bugs — a delivery quote that could silently show £0, a quantity stepper that couldn't decrease, dead back buttons after a deep link, a blank screen if fonts failed to load, mismatched order totals, an untrimmed basket key collision, and more — were fixed and are covered by the same test-and-verify pass. The complete line-by-line record is in Section 13.

3.2 — The design and responsiveness rebuild

In parallel, the mobile app's presentation layer was rebuilt around three goals: make it **reactive** (the interface visibly and immediately responds to touch, scroll and state changes), **fluid** (transitions use physically-modelled spring animation rather than abrupt cuts), and **responsive** (the same screens genuinely adapt across phone, tablet and desktop web rather than just stretching).

What changed, concretely

- **A shared design-token system** — typography scale, elevation/shadow levels, spring/duration/stagger motion constants, and named breakpoints (tablet, desktop, wide) — so every screen draws from one consistent source instead of hand-tuned values.
- **A responsive layout engine** (a single hook every screen reads) that decides column counts, card widths, gutters and content max-width from the live window size — phones get a bottom tab bar and single/double-column grids; tablets and desktop web get a left-hand navigation rail, wider grids, and split product-detail layouts.
- **Physically-modelled animation throughout** — a press-and-hover feedback wrapper used by every button and card; scroll-driven hero parallax with a header that fades and compacts as you scroll (built carefully so the compacted header can't intercept taps before it's actually visible); staggered entrance animations on list and grid items; a self-drawing progress line on the order-tracking timeline; haptic feedback on quantity and button interactions on native devices.
- **Interaction quality fixes found while redesigning** — a quantity-stepper button that only turns into a delete/trash icon in the basket (not on the product page, where reducing past one should simply stop at one); an internal component that was accidentally defined so it would remount and replay its own entrance animation on every keystroke, corrected to a stable element; a placeholder image technique that risked rendering as visual noise, removed before shipping; copy on a "Report product" action corrected so it doesn't imply a report was sent when no such backend endpoint exists yet.

How it was verified

The rebuilt web export was exercised in a real Chromium browser at four representative widths — 320px and 390px (phone), 834px (tablet), and 1440px (desktop) — with console error monitoring throughout. The only errors observed were expected image-loading failures caused by the test sandbox having no internet access to the hotlinked stock photo host; there were zero application errors.

Technical architecture

The monorepo

The codebase is a single repository organised as an npm-workspaces monorepo, following what's sometimes called the "Carpadi pattern": one shared package of types, tested business logic and the botanical dataset, consumed by both an owned backend and the mobile app. This matters commercially as much as technically — nothing here is locked into a third-party no-code platform, which is exactly the shape a business intends to grow, defend and eventually sell needs to be in.

```
Ewe ati Egbo/  
├─ shared/    @eweatiegbo/shared – domain types, pure business logic  
│              (search, compliance, routing), and the 46-herb botanical  
│              dictionary. Unit-tested; compiled once and imported by  
│              both the backend and the mobile app, so the rules for  
│              "is this claim allowed" are defined in exactly one place.  
├─ backend/   @eweatiegbo/backend – Fastify + PostgreSQL API. Catalogue,  
│              search, compliance-check and health endpoints today;  
│              order, payment and routing endpoints are the next layer.  
├─ mobile/    Expo (React Native) app – expo-router navigation, Supabase  
│              client for auth, a demo-data mode for development/screenshots.  
├─ supabase/  The full commerce & logistics database schema – sellers,  
│              cargo agents, corridors, shipments, payments, payouts,  
│              orders – as versioned SQL migrations.  
└─ docs/      Product requirements, market analysis, logistics model,  
              compliance rules, brand system, data model, and this  
              engineering record.
```

The backend

A Fastify (Node.js) API, chosen for its speed and first-class schema validation. Every request body and query parameter is validated against a strict schema (using Zod) before it touches any business logic, so malformed input is rejected with a clear error rather than crashing or executing unexpectedly. It talks to PostgreSQL through the standard `pg` driver, and can also run in a **demo mode** with no database configured at all — useful for local development, screenshots and this kind of walkthrough — falling back automatically when no database connection string is present, but only in non-production environments; a production deployment with a missing or placeholder secret now refuses to start rather than run insecurely.

The database — PostgreSQL via Supabase

The data layer does more than store rows; it actively enforces the platform's safety rules at the database layer itself, so a bug in a future API endpoint cannot bypass them:

- **Row-Level Security (RLS)** — by default, no one can read or write anything; explicit policies grant exactly the access each role needs (customers see the public catalogue and their own orders; sellers see their own listings and orders; admins see what admin duties require).
- **Column-level write guards** — database triggers block a user from editing their own role, verification status, payout account or a product's price/ commission directly, even if they find a way around the API layer.
- **Forward-only state machines** — a payment can only move through its states in one direction (e.g. it can never jump backwards from "captured" to "pending"), enforced by a trigger rather than trusted to application code.
- **Idempotent webhook handling** — every inbound Paystack webhook event is recorded once, keyed so a retried or maliciously replayed webhook cannot trigger a second payout.
- **Append-only, hash-chained audit logging** — every sensitive action is recorded in a log that cannot be edited or deleted after the fact, and each entry is cryptographically chained to the one before it, so tampering would be detectable.

The mobile app

Built with Expo SDK 54 and expo-router, so the same code produces an iOS app, an Android app and a web build. Authentication runs through Supabase's client library using the PKCE flow (the modern, more secure standard for mobile OAuth). Session tokens are stored in the device's encrypted keychain/keystore on native platforms and in browser session storage (cleared when the tab closes) on web — never in a plain, persistent, script-readable location. Animation runs on `react-native-reanimated`, which executes on the UI thread rather than JavaScript's main thread, which is what keeps gestures and scroll-linked effects smooth even under load.

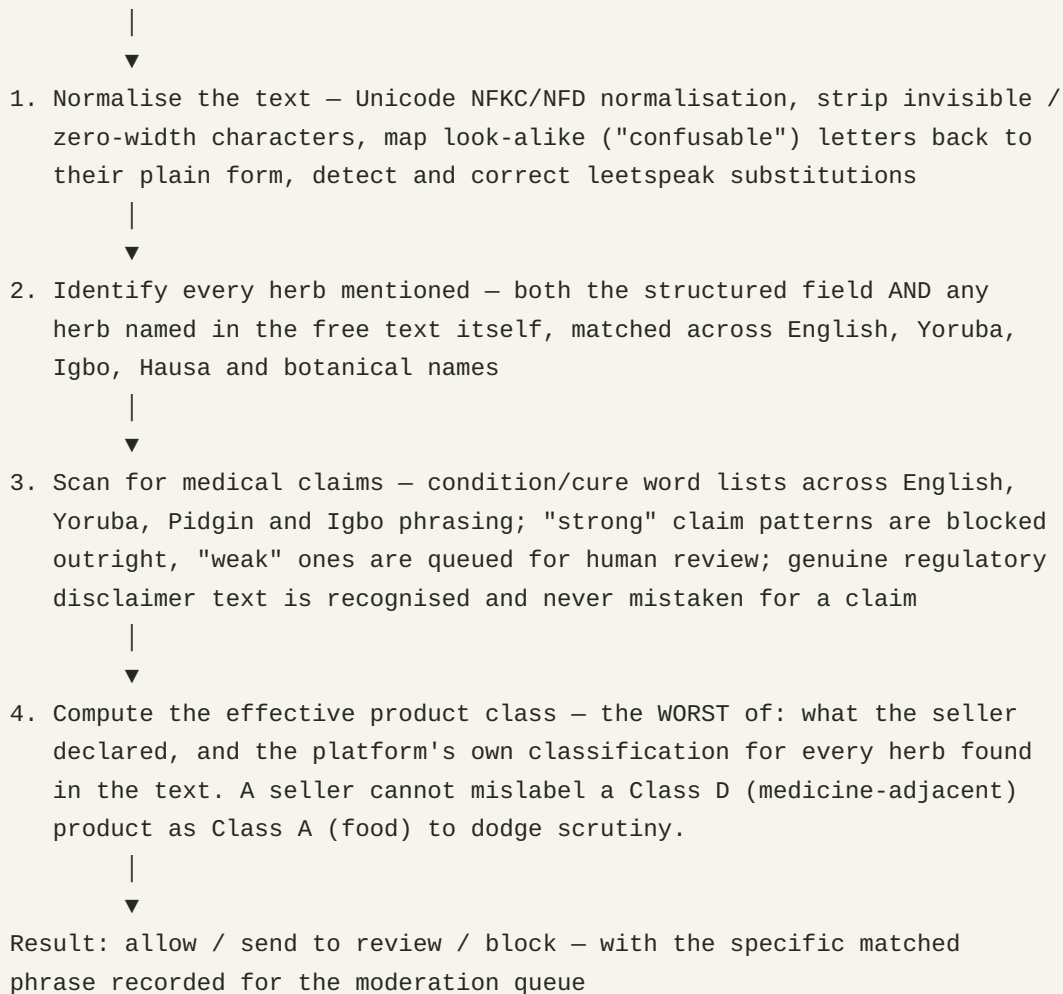
The compliance engine

Why this exists

UK law does not treat "herbal products" as one category. Depending on what a product is and what it is claimed to do, it falls under food/supplement law (Food Standards Agency), cosmetics law (Office for Product Safety and Standards), or medicines law (Medicines and Healthcare products Regulatory Agency) — and claiming a food or cosmetic product can "cure," "treat" or "prevent" a disease is exactly the line that turns an ordinary listing into a regulatory problem. The platform's entire value proposition to a seller is that they never have to learn these boundaries themselves — the platform enforces them automatically, before a listing can go live.

How a listing is checked

Listing text + declared class + herb reference



This rewrite is backed by **74 automated regression tests**, built directly from roughly 35 real bypass strings the audit surfaced (misspellings, vernacular phrasing, Cyrillic look-alikes, zero-width characters, emoji, leetspeak) plus a matching set of legitimate phrases the engine must *never* block — including the standard UK regulatory disclaimer text itself, so a compliant seller is never penalised for using the required wording.

Three herb flags under review

Three of the 46 catalogued botanicals had their safety flag tightened based on known chemistry, each carrying an explicit note pending confirmation from a UK regulatory adviser before it is treated as final:

Herb	Previous flag	New flag	Reason
Ebolo	Green	Amber	Contains pyrrolizidine alkaloids
Rere (senna seed)	Amber	Red	Toxic seed matter
Obi (kola)	Green	Amber	Caffeine content requires labelling

Payments

Payments run through **Paystack**, the leading payments processor for African markets, using its split-payment functionality to divide a single customer payment across subaccounts for the seller, the cargo agent who carried the shipment, and the platform's own commission — in one transaction rather than three.

What the database now supports

The database layer was rebuilt to make multi-party payouts *safe*, independent of which settlement policy is ultimately chosen:

- A payment can be split into any number of payout "legs" (seller, one or more cargo agents, platform), but the total of those legs can never exceed the original payment — enforced by a database trigger, not application logic that could have a bug.
- Every payment carries a currency, a foreign-exchange rate where relevant, and a verification timestamp, and moves through its states (created → verified → captured → settled, etc.) in one direction only.
- Every inbound payment webhook is deduplicated by provider reference, so a retried or replayed webhook from Paystack cannot cause a duplicate payout.

What is still an open decision, not a bug

Split-at-charge vs. collect-then-transfer. The audit's central recommendation was to move from paying sellers and cargo agents *at the moment of charge* to **collecting the full payment first and transferring payouts only after delivery is confirmed**, with an optional holdback period for dispute protection. This is a business-policy decision, not an engineering one — the database now supports either model safely — and the actual checkout integration with Paystack has not yet been built (the checkout button in the app is currently a placeholder). This is flagged explicitly here rather than left implicit, because it is the single highest-leverage decision left before the platform can process a real transaction.

Customers

Primary audience

UK-based buyers of African and Caribbean-heritage botanicals — a population of more than 2.5 million people of African and Caribbean descent in the UK, plus a wider wellness-curious audience interested in traditional and natural products. They currently source these products through informal, low-trust channels — WhatsApp groups, Instagram sellers, Etsy listings under inconsistent names, or travellers bringing goods back in suitcases — with no verification, no consistent quality, and no buyer protection.

What the platform gives them that the status quo doesn't

- **Search in their own language.** A customer can type "ewuro" (Yoruba), "onugbu" (Igbo), or "bitter leaf" (English) and reach the same, correctly identified product — instead of the ambiguous, sometimes-wrong guessing that informal sellers and even the platform's own earlier search logic were prone to.
- **Verified sellers** with a real trust panel reflecting their actual, current verification status — not a hard-coded green tick shown to everyone regardless of whether they've actually been checked.
- **An honest delivery quote before paying** — corridor, carrier, price and an ETA window shown upfront, never a silent £0 placeholder for a route that doesn't actually exist, and checkout is disabled outright if any basket item has no deliverable route.
- **End-to-end tracking** on a single, shared timeline — from the origin hub in Lagos, Accra or wherever the item started, through customs, to the customer's UK address — with proof required before a shipment can be marked "delivered."

Clients — sellers, cargo agents & admin

Sellers — African producers and UK diaspora shops

Two seller populations, both served by the same tools: producers and small businesses based in Africa who want reach into the UK diaspora market without building their own export and compliance capability from scratch, and UK-based diaspora shops who already serve their local community and want a wider, structured storefront. Both get: guided onboarding and identity/business verification, a structured listing wizard with automatic compliance checking (so a listing is checked before it can ever reach a customer, not after a complaint), order management, payouts, and a choice between using the platform's own cargo network or arranging their own UK-domestic fulfilment.

Cargo agents — the supply side of the logistics network

Verified carriers — starting with the platform's own in-house cargo arm ("agent zero") and expanding to onboarded third-party freight and courier operators — who serve one or more Africa→UK corridors. Each agent publishes the routes they operate (corridor, rate model, transit time, capacity) and is rated on on-time performance, damage/loss rate and how well they handle exceptions; routing favours better-performing agents automatically, and consistently poor performers receive less volume or are suspended. Onboarding requires identity/business verification, proof of corridor experience, insurance/handling declarations, and a Paystack payout subaccount.

Admin — the platform operator

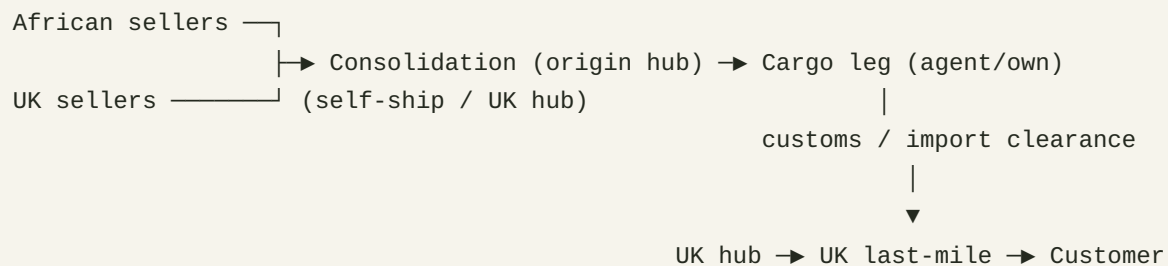
The internal team (or eventually a small ops function) responsible for seller and product verification, moderating anything the compliance engine sends to review, keeping import and corridor rules current, overseeing routing decisions, and handling orders and disputes. Every privileged action they take is now recorded in the tamper-evident audit log described in Section 4.

Logistics — the Africa→UK freight network

Why this is the real asset

Everyone who has tried to build something like this has built a storefront — that's the easy 20% of the problem. The unsolved 80% is cross-border logistics for a fragmented, low-trust supply chain: consolidating small parcels from many independent African sellers, clearing UK food/botanical import rules, and delivering to the door at a price that still works commercially. Whoever actually operates that rail owns the category, because a competitor cannot simply copy a UI to compete with it.

The flow



Core entities

Entity	What it is
Corridor	An origin→destination lane — e.g. Lagos→London, Accra→London
Cargo agent	A verified carrier (the platform's own arm counts as "agent #0") serving one or more corridors at published rates and capacity
Route	A specific cargo agent's offer on a corridor — price model, transit time, handling terms
Consignment	A batch a cargo agent physically carries — may consolidate items from many different sellers and orders travelling the same corridor at once
Shipment	The customer-facing unit — the items from one order, travelling together, with their own status timeline

An order fans out into one or more shipments; shipments are grouped into consignments for the physical cargo leg; a consignment is carried by exactly one cargo agent over one corridor.

Three fulfilment routes

1. **Platform cargo (Africa-origin)** — the default for African sellers: drop at an origin hub, the platform consolidates, its own or a partner agent carries to the UK, customs clearance, then UK last-mile delivery.
2. **Third-party cargo agent** — the same shape, with the cross-border leg carried by an onboarded agent the platform routes to and pays.
3. **UK direct (self-ship)** — for UK-based sellers, a purely domestic shipment via a standard UK courier, with no cross-border leg at all.

Where the margin comes from

Many small parcels from many small, independent sellers travelling the same corridor are consolidated into one consignment, which drops the per-unit shipping cost — and the platform keeps the spread between what it pays the carrying agent and what it charges the customer. This consolidation efficiency compounds directly with order volume, which is precisely why the network becomes harder to replicate the more it's used.

Tracking and proof of delivery

Every shipment moves through an explicit status timeline — created, at origin hub, in transit, customs, at UK hub, out for delivery, delivered (or exception) — visible identically to the customer and the seller. Critically, a shipment can no longer be marked "delivered" by a cargo agent's say-so alone: the database now requires actual evidence (a proof-of-delivery event) before that final status can be recorded, closing a gap where an agent could previously claim delivery — and trigger a payout — with nothing to back it up.

Business model

The platform earns on **two independent lines per order** — product and logistics — which is what makes the unit economics fundamentally different from a standard marketplace commission alone:

Revenue line	Mechanism
Product commission	A percentage of each product sale (indicatively 8–12%)
Logistics margin	The largest line: markup on freight, plus the spread between what a cargo agent is paid and what the customer is charged, improved further by consolidation efficiency
Cargo-agent platform fee	A fee on legs routed to third-party agents rather than the platform's own cargo arm
Seller subscriptions / featured listings (<i>later</i>)	Recurring revenue from sellers wanting greater visibility
Data & wholesale (<i>later</i>)	Aggregated demand data and B2B bulk corridors

Why this is defensible

Layer	How hard it is to copy
Logistics network	Very hard. Onboarded cargo agents, proven corridors, customs know-how and consolidation volume compound with scale and are slow and expensive to rebuild from zero.
Compliance rail	Hard. Product-class rules and UK import/ food-safety handling are done <i>for</i> the seller automatically — a barrier that keeps informal competitors informal.
Multilingual botanical data	Medium. The vernacular-to- botanical mapping across Yoruba, Igbo, Hausa and English is a proprietary dataset and a long-term search/SEO advantage.
Trust and verified supply	Medium. Two-sided liquidity plus verified sellers is a cold-start advantage once the marketplace is seeded.

Expansion path

Beachhead: West African and Caribbean UK diaspora. Then additional African origin corridors (Ghana, Kenya, South Africa), deeper UK demand, and eventually replicating the same operating model into US and EU diaspora corridors — the logistics network and compliance rail are the reusable core assets that travel to each new lane.

Verification & evidence — nothing here is "trust me"

Every fix claimed in this document was verified against a running system, not just read and approved on paper:

- **74 automated unit tests** for the search and compliance logic — every test passing, run both in an isolated build environment and re-run directly on the production device.
- **A live PostgreSQL 16 instance**, with a hand-built stub of Supabase's authentication schema, used to apply both database migrations and then run **16 simulated attacks** — attempting self-promotion to admin, duplicate webhook payments, capturing an unverified payment, moving a payment backwards through its states, paying out more than a payment was worth, cascading-deleting a paid order, setting a negative or over-100% commission, attaching another customer's address to an order, marking a shipment "delivered" with no evidence, tampering with the audit log, reading another customer's private data, writing directly to a protected table, and a seller attempting to self-verify or hijack their own payout account. **Every attack was correctly blocked**, while legitimate operations — editing one's own profile, delivering with proper evidence, reading the public catalogue — succeeded normally.
- **Live HTTP probes** against the running Fastify server, confirming clean 400 responses on bad input, a restricted CORS policy, and correct rate limiting.
- **npm audit on the backend: zero known vulnerabilities** after the dependency clean-up.
- **Browser testing** of the production web export in real Chromium, at 320px, 390px, 834px and 1440px, with console monitoring throughout.

What's next — decisions and work still ahead

The following are deliberately **not** presented as finished, because they either require a decision only the business can make, or fall outside what code alone can resolve:

1. The payment settlement model

Collect-then-transfer with a holdback period is the recommended model; the database now supports it safely, but the choice itself, and the actual Paystack checkout integration, are still to be built. This is the single most important next step before real money can move through the platform.

2. Final sign-off on three herb safety flags

Ebolo, Rere and Obi (kola) were re-flagged based on known chemistry (Section 5) but each is explicitly marked pending confirmation from a qualified UK regulatory adviser.

3. Applying the new database migration to the live project

The hardening migration has been thoroughly applied and attack-tested against a local test database; it has not yet been applied to the live/production Supabase project.

4. Rebuilding the public demo site

The GitHub Pages demo export needs to be rebuilt with the corrected build script (which now always clears the bundler cache) so it reflects the redesigned, hardened app rather than the previous build.

5. UK GDPR paperwork

Handling UK customer data from outside the UK requires a data-transfer agreement and risk assessment, a UK Article 27 representative, and data-processing agreements with onboarded cargo agents. This is legal and administrative work, explicitly out of scope for an engineering pass.

Full engineering fix log

Source: `docs/security-audit-2026-09-23.md`, branch `fix/security-and-design`.

Medium-severity items — all addressed

- Missing CHECK constraints allowing negative prices/quantities and commission over 100% — **fixed** with enum-style and range constraints across every relevant column.
- No currency column, risking a ~1500x unit error (e.g. Naira kobo mistaken for British pence) — **fixed**, currency added to 9 tables.
- `payments.provider` defaulting to the wrong payment processor — **fixed**, forced to the correct provider.
- An order's delivery address could belong to a different customer entirely — **fixed** with a database trigger validating ownership.
- Backend returning raw 500 errors with internal schema details on bad input — **fixed** with a proper error handler.
- CORS reflecting any requesting origin — **fixed**, restricted to an explicit allowlist.
- Rate limiting misconfigured behind a reverse proxy — **fixed** with a correct trust-proxy configuration.
- Server defaulting to development mode if an environment variable was simply unset — **fixed**, production is now explicit and secrets are validated.
- Vernacular search returning the wrong herb for ambiguous queries, missing accented characters entirely, or a single-letter query matching almost the whole catalogue — **fixed**, rewritten to refuse to guess rather than return a wrong answer.
- Compliance engine false-positives blocking innocuous phrases like "sweet treats" or the standard regulatory disclaimer itself — **fixed**, with an explicit allow-list and disclaimer recognition.
- Trust panel showing a hard-coded "Verified" tick regardless of actual status — **fixed**, now reflects the real, current seller state.
- A race condition in the mobile app's authentication state that could leave the loading spinner running forever — **fixed**, with a safety timeout.
- Database seed script not running in a transaction, and not cleaning up stale restricted-ingredient rows for downgraded herbs — **fixed**.
- Cargo agents able to mark a shipment "delivered" with no proof — **fixed**, proof is now required by a database trigger.
- Incomplete `.gitignore` coverage for secret file types — **fixed**, broadened to cover certificates, keys and cloud service-account files.
- A fresh clone of the repository breaking the backend because a required build step was missing — **fixed** with an automatic build step on install.

Mobile functional bugs — all 12 fixed

1. Missing or unknown delivery corridor silently showing a £0 quote

2. No way to decrease quantity or remove a single basket item
3. Back and close buttons doing nothing after a deep link or web page refresh
4. "Product not found" being a dead end with no way forward
5. The app staying blank indefinitely if a font failed to load
6. The error-boundary screen having no retry option and exposing raw error text
7. Sample order totals not matching the app's own pricing logic
8. Unknown product IDs lingering in the basket, showing a £0 "empty" state
9. Dead buttons across the app — Shop Now, category tiles, tradition filters, Report product, account rows
10. Icon-only buttons with no accessibility labels; an unannounced cart count
11. Fixed-width cards overflowing on narrow (320px) phones; a basket list key collision
12. Signing out not clearing the shopping basket

Checked and already clean (no action needed)

No secrets found in the repository or its git history. Search and compliance regular expressions are not vulnerable to denial-of-service attacks (processed 1MB of adversarial input in 12 milliseconds). No prototype-pollution vulnerability on the herb lookup endpoint. Security headers (Helmet) are active. No source maps shipped in the production build. The custom 404 page has no open-redirect vulnerability. Marketing copy contains no disallowed health claims. All mock/demo listings pass the compliance scanner cleanly.

Housekeeping

The web export previously had no Content-Security-Policy meta tag (and a misspelled attribute meant the browser ignored the one that was there); stock photography and web fonts were being hotlinked from third-party hosts rather than served from the platform's own domain; the pitch document previously stated ownership of a freight network and proven corridors in the present tense rather than as a forward-looking plan; the root project's mobile-app npm script was broken because the mobile app isn't part of the workspace configuration; and dependency-vulnerability auto-fixing was deliberately avoided in the mobile app because it would have downgraded a core navigation library — the flagged mobile vulnerabilities are almost entirely in build-time tooling, not the shipped app.

Ewe ati Egbo — leaves and roots, moved with trust.

This document was prepared as a complete, evidence-backed account of the work completed on branch `fix/security-and-design`, committed alongside the code it describes.